MannyPay API Documentation (v1)

Welcome to the MannyPay API. Our API allows merchants to accept payments (Pay-In) and send funds (Disbursement).

1 Base URL

All API requests should be made to the base URL:

```
https://mannypaytest.com/v1
```

2 Authentication

The MannyPay API uses **API Keys** to authenticate requests. You can view and manage your API keys in the MannyPay Merchant Dashboard.

Your API requests must include your **Secret Key** in the `Authorization` header using the `Bearer` scheme.

```
Authorization: Bearer SECRET_KEY
```

> **WARNING:** Your `SecretKey` carries the power of your account. Do not share it, embed it in client-side code, or commit it to GitHub. Keep it strictly on your backend server.

3 Idempotency

To safely retry requests without accidentally performing the same operation twice (e.g., double-charging a customer or sending two disbursements), the MannyPay API supports idempotency.

For all `POST` requests, you **must** include an `Idempotency-Key` header with a unique string (like a UUID) generated by your system.

```
Idempotency-Key: 123e4567-e89b-12d3-a456-426614174000
```

If your connection drops and you retry the request with the same key, MannyPay will bounce back the exact same response it generated the first time, without processing the transaction again.

4 Pay-in

Pay-in is the process of collecting money from a user, such as generating a dynamic QRPh code.

4.1 Create a Pay-in

Creates a new pay-in and generates a QR code for the customer to scan.

Endpoint: `POST /pay-in`

Request Body:

```
{
  "merchant_reference": "order_10293",
  "amount": 1500,
  "payment_method": "qrph",
  "description": "Payment for Order 10293",
  "email": "juandelacruz@gmail.com",
  "return_url": "https://merchantsite.com/tran?ref=123"
}
```

Response (200 OK):

```
{
  "ref_id": "MP-QRPH-20260316103343043",
  "amount": 1500,
  "status": "pending",
  "payment_method": "qrph",
  "qr_code_string": "00020101021228760011ph.ppmi.p2m...",
  "qr_code_image_url": "https://mannypaytest.com/qr.html?qr=...",
  "created_at": "2026-03-16T10:33:43.04"
}
```

4.2 Retrieve a Pay-in

Checks the status of an existing Payment.

Endpoint: `GET /pay-in/{ref_id}`

Response (200 OK):

```
{
  "ref_id": "MP-QRPH-20260316103343043",
  "amount": 1500.00,
  "fee": 10.00,
  "status": "succeeded",
  "payment_method": "qrph",
  "created_at": "2026-03-16T10:33:43.04"
}
```

4.3 Cancel a Pay-in

Cancels a pending pay-in transaction.

Endpoint: `POST /pay-in/cancel/{ref_id}`

Response (200 OK):

```
{
  "message": "Transaction successfully cancelled.",
  "ref_id": "MP-QRPH-20260316103343043",
  "status": "cancelled"
}
```

5 Disbursement

Disbursement allow you to send money from your MannyPay to external bank accounts or e-wallets via InstaPay or Pesonet.

5.1 Create a Disbursement

Sends funds to a destination account. InstaPay supports up to 50,000 PHP instantly, while Pesonet handles larger amounts but takes up to 24 hours.

Endpoint: `POST /disbursement`

Request Body:

```
{
  "merchant_reference": "payout_9921",
  "amount": 25000,
  "destination": {
    "bank_code": "0053",
    "account_number": "003920251444",
    "account_name": "Juan Dela Cruz"
  },
  "channel": "instapay"
}
```

(Note: `channel` must be either `"instapay"` or `"pesonet"`)

Response (200 OK):

```
{
  "ref_id": "MP-INSTA-20260316111302840",
  "amount": 25000,
  "fee": 10.00,
  "net_amount": 25010.00,
  "status": "processing",
  "estimated_arrival": "2026-03-16T03:18:03.42"
}
```

5.2 Retrieve a Disbursement

Checks the status of an existing disbursement.

Endpoint: `GET /disbursement/{id}`

Response (200 OK):

```
{
  "id": "MP-INSTA-20260316111302840",
  "amount": 25000.00,
  "fee": 10.00,
  "netAmount": 25010.00,
  "status": "successful",
  "created_at": "2026-03-16T11:13:02.84"
}
```

Possible Statuses: `processing`, `succeeded`, `failed`

5.3 Scan QR

Parses a QRPh string to extract payment details like bank code, account number, and amount (if present).

Endpoint: `POST /disbursement/scan/qr`

Request Body:

```
{
  "qr_string": "00020101021228530011ph.ppmi.p2m..."
}
```

Response (200 OK):

```
{
  "amount": 150.00,
  "bank_code": "7018",
  "account_number": "639989553376",
  "account_name": "RHN MERCHANT CORP",
  "type": "P2M",
  "qr_identifier": "91131"
}
```

5.4 Pay Scanned QR

Executes a payment using the details obtained from a scanned QR code string.

Endpoint: `POST /disbursement/scan/pay`
Request Body:

```
{
  "merchant_reference": "scan_pay_102",
  "amount": 150,
  "destination": {
    "bank_code": "7018",
    "account_number": "639989553376",
    "account_name": "Juan Dela Cruz"
  },
  "type": "P2M",
  "qr_identifier": "91131"
}
```

Response (200 OK):

```
{
  "ref_id": "MP-INSTAQR-20260316113703908",
  "amount": 150,
  "fee": 10.00,
  "net_amount": 160.00,
  "status": "processing",
  "estimated_arrival": "2026-03-16T11:42:03.90"
}
```

6 Balances & Settings

6.1 Retrieve Wallet Balance

Check the available balance in your MannyPay account.

Endpoint: `GET /balance`
Response (200 OK):

```
8870.00
```

6.2 Get InstaPay Bank codes

Retrieve the list of valid bank codes for InstaPay disbursements.

Endpoint: `GET /bank_codes/instapay`
Response (200 OK):

```
[
  {
    "code": "7021",
```

```
    "name": "Alipay / Lazada Wallet"
  },
  {
    "code": "0102",
    "name": "Asia United Bank Corporation"
  }
]
```

6.3 Get Pesonet Bank codes

Retrieve the list of valid bank codes for Pesonet disbursements.

Endpoint: `GET /bank_codes/pesonet`

Response (200 OK):


```
[
  {
    "code": "0196",
    "name": "AGRIBUSINESS BANKING CORP. - A RURAL BANK"
  },
  {
    "code": "0036",
    "name": "AL-AMANAH ISLAMIC BANK"
  }
]
```


7 Webhooks (Events)

Instead of constantly polling the `GET` endpoints to check if a transaction succeeded, MannyPay can send push notifications (Webhooks) directly to your server whenever an event occurs.

7.1 Receiving Webhooks

Configure your `WebhookUrl` in the Dashboard. When a transaction status changes, we will send an HTTP `POST` to your URL with the following JSON schema:

Pay-in event:


```
{
  "event_id": "evt_ab12cd34ef56",
  "type": "payment.succeeded",
  "created_at": "2026-03-16T12:35:00Z",
  "data": {
    "id": "MP-QRPH-20260316103343043",
    "merchant_reference": "order_10293",
    "amount": 1500.00,
    "status": "succeeded"
  }
}
```


Disbursement event:


```
{
  "event_id": "evt_78gh90ij12kl",
  "type": "disbursement.succeeded",
  "created_at": "2026-03-16T13:45:10Z",
  "data": {
    "id": "MP-INSTA-20260316111302840",
    "merchant_reference": "payout_9921",
```

```
    "amount": 25000.00,
    "status": "succeeded"
  }
}
```

Event types include `payment.succeeded`, `payment.failed`, `disbursement.succeeded`, and `disbursement.failed`.

7.2 Securing Webhooks (Signature Verification)

To ensure the webhook is truly from MannyPay and hasn't been tampered with, we sign every request using a Hash-based Message Authentication Code (HMAC) with SHA-256.

1. MannyPay generates a hash taking the **raw JSON payload body** and your **WebhookSecret** (found in your Dashboard). 2. The resulting hash is placed in the `MannyPay-Signature` header.

To verify in your backend (Example in Node.js):

```
const crypto = require('crypto');

app.post('/webhook', (req, res) => {
    const signature = req.headers['mannypay-signature'];
    const payload = JSON.stringify(req.body); // Ensure raw string matches
    exactly

    const expectedSignature = crypto
        .createHmac('sha256', process.env.WEBHOOK_SECRET)
        .update(payload)
        .digest('hex');

    if (signature === expectedSignature) {
        // Safe to process!
        console.log("Valid webhook received!", req.body.type);
        res.status(200).send('OK');
    } else {
        res.status(401).send('Invalid Signature');
    }
});
```